Monaco Configuration-as-Code

> **Series:** AUTOM | **Notebook:** 3 of 8 | **Created:** January 2026

Table of Contents

1. Introduction
2. Getting Started
3. Project Structure
4. Configuration Files
5. Common Operations
6. Advanced Features
7. Next Steps

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Monaco CLI	Installed via GitHub releases or brew
API Token	Token with `settings.read`, `settings.write`, `ReadConfig`, `WriteConfig`
Tenant URL	Your Dynatrace SaaS tenant URL

Learning Objectives

By the end of this notebook, you will:

- Understand Monaco's configuration-as-code model
- Know how to download existing configurations
- Be able to create and deploy configurations via YAML
- Handle multi-environment deployments

---

1. Introduction

Monaco (Monitoring as Code) is Dynatrace's official CLI tool for configuration management. It uses YAML files to define configurations and supports version control, CI/CD integration, and multi-environment deployments.

Why Monaco?

Benefit	Description
Version Control	Track all changes in Git
Repeatability	Deploy same config to multiple environments
Review Process	Use pull requests for config changes
Disaster Recovery	Quickly restore configurations
Documentation	YAML files serve as living documentation

Monaco vs Direct API

Aspect	Monaco	Settings API
Format	YAML files	JSON payloads
State management	Implicit (by name)	Manual tracking
Multi-tenant	Built-in support	Custom scripting
Dry run	Yes (`--dry-run`)	No
Delete orphans	Yes (`delete` command)	Manual

---

2. Getting Started

Installation

macOS (Homebrew):

```
brew install dynatrace-oss/monaco/monaco
```

Linux/Windows:

```
# Download from GitHub releases
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
chmod +x monaco
```

Verify installation:

```
monaco version
```

Environment Setup

Create environment variables for authentication:

```
export DT_TENANT_URL="https://{tenant-id}.live.dynatrace.com"
export DT_API_TOKEN="dt0c01.XXXXXXXX.YYYYYYYY"
```

Your First Download

Download all configurations from your tenant:

```
monaco download \
  --environment-url "$DT_TENANT_URL" \
  --api-token "$DT_API_TOKEN" \
  --output-folder ./downloaded-config
```

Download specific configuration types:

```
monaco download \
  --environment-url "$DT_TENANT_URL" \
  --api-token "$DT_API_TOKEN" \
  --output-folder ./downloaded-config \
  --api builtin:management-zones,builtin:tags.auto-tagging
```

---

3. Project Structure

Recommended Layout

```
dynatrace-config/
├── manifest.yaml              # Project manifest
├── environments/
│   ├── development.yaml       # Dev environment config
│   ├── staging.yaml           # Staging config
│   └── production.yaml        # Production config
└── projects/
    └── my-project/
        ├── management-zones/
        │   └── config.yaml
        ├── auto-tagging/
        │   └── config.yaml
        └── alerting-profiles/
            └── config.yaml
```

Manifest File

The `manifest.yaml` defines your project:

```yaml
manifestVersion: 1.0

projects:
  - name: my-project
    path: projects/my-project

environmentGroups:
  - name: default
    environments:
      - name: development
        url:
          type: environment
          value: DT_DEV_URL
        auth:
          token:
            type: environment
            value: DT_DEV_TOKEN
      - name: production
        url:
          type: environment
          value: DT_PROD_URL
        auth:
          token:
            type: environment
            value: DT_PROD_TOKEN
```

---

4. Configuration Files

Basic Configuration Structure

```yaml
configs:
  - id: production-zone
    config:
      name: Production
      template: management-zone.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment
```

Management Zone Example

config.yaml:

```
configs:
  - id: production-mz
    config:
      name: Production
      template: production-mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment

  - id: staging-mz
    config:
      name: Staging
      template: staging-mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment
```

production-mz.json:

```
{
  "name": "{{ .name }}",
  "rules": [
    {
      "type": "SERVICE",
      "enabled": true,
      "conditions": [
        {
          "key": {
            "type": "STATIC",
            "attribute": "SERVICE_TAGS"
          },
          "comparisonInfo": {
            "type": "TAG",
            "operator": "EQUALS",
            "value": {
              "context": "CONTEXTLESS",
              "key": "environment",
              "value": "production"
            },
            "negate": false
          }
        }
      ]
    }
  ]
}
```

Auto-Tagging Example

config.yaml:

```
configs:
  - id: application-tag
    config:
      name: Application
      template: application-tag.json
    type:
      settings:
        schema: builtin:tags.auto-tagging
        scope: environment
```

application-tag.json:

```
{
  "name": "{{ .name }}",
  "rules": [
    {
      "type": "SERVICE",
      "enabled": true,
      "valueFormat": "{Service:DetectedName}",
      "propagationTypes": ["SERVICE_TO_PROCESS_GROUP_LIKE"],
      "conditions": []
    }
  ]
}
```

Using Variables

Variables allow environment-specific values:

config.yaml:

```
configs:
  - id: alerting-profile
    config:
      name: "{{ .Env.ENVIRONMENT_NAME }} Alerting"
      template: alerting.json
      parameters:
        severity_level: "{{ .Env.ALERT_SEVERITY }}"
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

---

5. Common Operations

Deploy Configurations

Deploy to all environments:

```
monaco deploy manifest.yaml
```

Deploy to specific environment:

```
monaco deploy manifest.yaml --environment production
```

Dry run (validate without applying):

```
monaco deploy manifest.yaml --dry-run
```

Validate Configurations

Check YAML syntax and schema validity:

```
monaco validate manifest.yaml
```

Delete Configurations

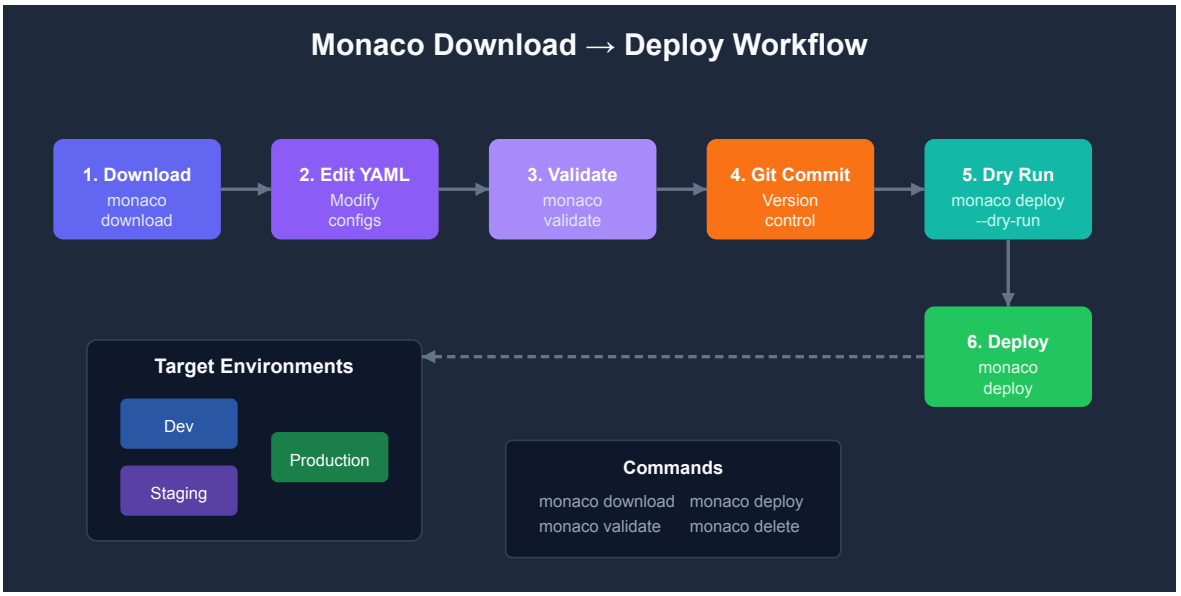
Remove configurations not in your project (orphans):

```
monaco delete manifest.yaml --file delete.yaml
```

delete.yaml:

```
delete:
  - type: builtin:management-zones
    id: old-management-zone-id
```

Download vs Deploy Workflow



---

6. Advanced Features

Configuration Dependencies

Reference other configurations:

```
configs:
  - id: my-alerting-profile
    config:
      name: Production Alerting
      template: alerting.json
      parameters:
        management_zone_id: "{{ .management-zones.production-mz.id }}"
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

Conditional Configurations

Skip configurations in certain environments:

```yaml
configs:
  - id: production-only-config
    config:
      name: Production Monitor
      template: monitor.json
      skip:
        environments:
          - development
          - staging
    type:
      settings:
        schema: builtin:synthetic.http
        scope: environment
```

Environment-Specific Overrides

```yaml
configs:
  - id: alerting
    config:
      name: Alerting Profile
      template: alerting.json
      parameters:
        delay_minutes: 5
      overrides:
        - environments:
            - production
          override:
            parameters:
              delay_minutes: 1
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

Grouping Configurations

Organize related configs in groups:

```
configs:
  - id: web-app-configs
    group: web-application
    config:
      name: Web App Management Zone
      template: mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment

  - id: web-app-alerting
    group: web-application
    config:
      name: Web App Alerting
      template: alerting.json
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

Deploy only a specific group:

```
monaco deploy manifest.yaml --group web-application
```

Best Practices

Practice	Description
Use meaningful IDs	IDs should describe the config purpose
Modular structure	Separate configs by domain (alerting, monitoring, etc.)
Environment variables	Never hardcode tokens or URLs
Version control	Commit all changes with meaningful messages
Dry run first	Always validate before applying
Document parameters	Comment complex configurations

Common Issues and Solutions

Issue	Cause	Solution
"Config already exists"	Duplicate ID	Use unique IDs or download first
"Schema not found"	Typo in schema ID	Check exact schema name in API
"Template not found"	Wrong path	Use relative path from config.yaml
"Validation failed"	Invalid JSON	Check JSON syntax in template

---

7. Next Steps

When to Consider Terraform

Scenario	Tool Choice
Config-only management	Monaco
Part of larger IaC stack	Terraform
Need state file management	Terraform
Drift detection required	Terraform
Simple GitOps workflow	Monaco

Continue the Series

Next Notebook	Focus
AUTOM-04: Terraform Provider	Infrastructure-as-code approach

Additional Resources

- Monaco Documentation
- Monaco Examples
- Configuration Schema Reference

Summary

In this notebook, you learned:

- How to install and configure Monaco
- Project structure and manifest configuration
- Creating and deploying configurations via YAML
- Advanced features like dependencies, overrides, and groups

> **Key Takeaway:** Monaco is ideal for teams wanting GitOps-style configuration management. It provides the right balance of simplicity and power for most Dynatrace automation needs.

Continue to AUTOM-04: Terraform Provider to learn infrastructure-as-code patterns.
